

Lecture 24: Attention

How tokens share meaning

GenAI chapter. Parts adapt LMU I2DL (CC BY 4.0) and 3Blue1Brown, “Deep Learning” Ch. 5–6 (CC BY-NC-SA).

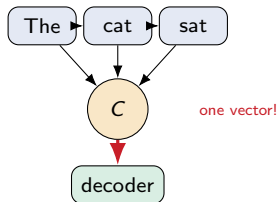
Last chapter ended on a cliffhanger

In L21, the encoder read a whole sentence and crushed it into **one** context vector C . The decoder then wrote the translation from C alone.

For a short sentence, fine. For a long one, C is a bottleneck – the ending forgets the beginning.

Even the LSTM could not fix it: the memory is still one fixed vector.

The question we left open: *what if the decoder could look back at **all** the encoder states, not just the last one?*



The fix, in one idea

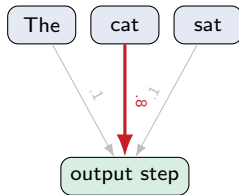
Predict: how would *you* fix the bottleneck?

The fix, in one idea

Predict: how would *you* fix the bottleneck?

Don't force everything through one vector.
At **each** step of writing the output, let the decoder **look back over every input state** and take a *weighted average* – heavier weight on the states that matter right now.

That weighted look-back *is* **attention**. The whole chapter unpacks this one sentence – and then takes it much further.



weights sum to 1; here the output leans on “cat”

Words are vectors; directions carry meaning

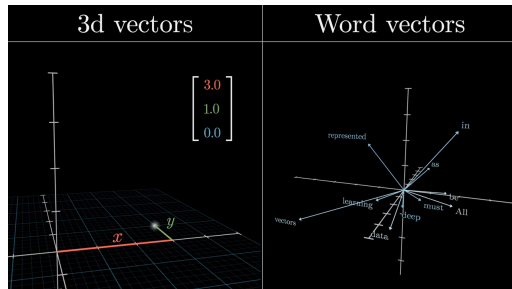
Attention moves a word's vector around a meaning-space. First, recall what that space is.

Recall: a token is a vector

Every token is mapped to a vector – its **embedding** – a point in a high-dimensional space (GPT-3: 12,288 dims). You met this in L21: we plotted word vectors in 2D.

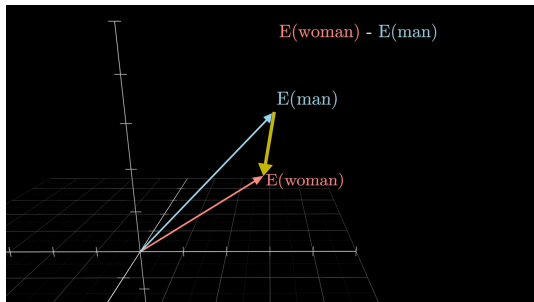
The key idea: **directions in this space carry meaning**. Nearby vectors are similar words.

A word starts as a lookup – *context-free*. The whole job of the network is to enrich it.



Screenshot: 3Blue1Brown, CC BY-NC-SA

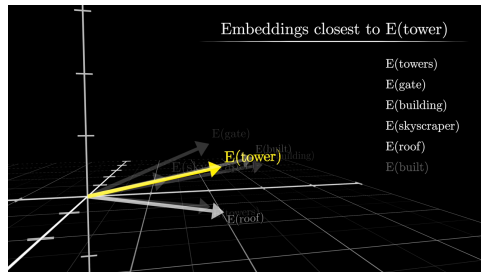
Directions carry meaning



The famous one: $\text{king} - \text{man} + \text{woman} \approx \text{queen}$. A single *direction* encodes “gender”; another “royalty.” (word2vec:

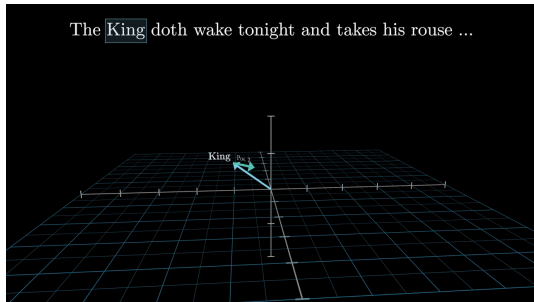
Mikolov et al., 2013)

Screenshot: 3Blue1Brown, CC BY-NC-SA



The **dot product** measures how much two vectors align. **Remember this** – it is the engine of attention.

Embeddings soak up context



Screenshot: 3Blue1Brown, CC BY-NC-SA

The same word means different things: “a machine learning **model**” vs. “a fashion **model**.” The starting embedding of *model* is identical in both.

Something has to **pull** that generic vector toward the right meaning, using the surrounding words.

That pull is exactly what attention does: move each embedding to a context-specific point in meaning-space.

The attention idea

Score, softmax, weighted sum – the whole mechanism in three moves.

Attention = a weighted look-back

At each output position, attention does three things:

1. **Score** every other position for relevance (how much should I look there?).
2. **Softmax** the scores into weights that are positive and sum to 1.
3. **Weighted sum**: blend the other positions' information by those weights.

That's it – a **content-based average**. The rest of this deck is just *how* we compute the scores and *what* we blend. No new calculus: softmax and weighted sums are L14.

You can see it

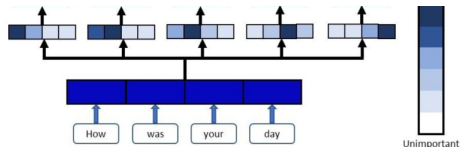


Figure: An illustration of a machine translation task using an encoder-decoder model with an attention mechanism. The attention weights at each time-step of the decoding/translation process indicate which parts of the input sequence are most relevant. There are 4 attention weights because there are 4 encoder states (Loye, 2019).



A woman is throwing a frisbee in a park.



A stop sign is on a road with a mountain in the background.

image captioning: the attention mechanism tells the

Translation: each output word's attention over the input words
– the model learns the alignment itself. (Bahdanau, Cho & Bengio, 2015 – the first attention)

Captioning: attention over *pixels* while writing each word. (Xu et al., 2015)

Same skeleton – score, softmax, weighted sum – whether the “positions” are source words or image patches. Next: let the model *learn* what to score.

The leap: drop the RNN

If attention gives direct access to everything, the sequential RNN is dead weight.

Why keep the RNN at all?

Predict: attention already lets any output position pull from any input position, *directly*.
So why keep the slow, sequential RNN underneath – the part that forces us to process one token at a time?

Why keep the RNN at all?

Predict: attention already lets any output position pull from any input position, *directly*. So why keep the slow, sequential RNN underneath – the part that forces us to process one token at a time?

You don't. Rip out the recurrence, keep only attention. That is the 2017 paper whose title says it all: **“Attention Is All You Need.”**

(Vaswani et al., 2017)

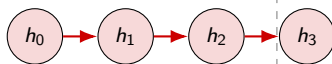
Stack these attention layers and the vectors get refined again and again – meaning deepens with depth.



Screenshot: 3Blue1Brown, CC BY-NC-SA

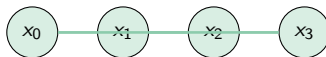
Sequential vs. all-to-all

RNN / LSTM



sequential: $O(n)$ steps to connect ends

Transformer



all-to-all: $O(1)$ – direct

Three wins: **parallel** (all tokens at once) · **direct access** (any token to any other) · **contextual** (representations change with context).

Why context matters

American shrew mole

One mole of carbon dioxide

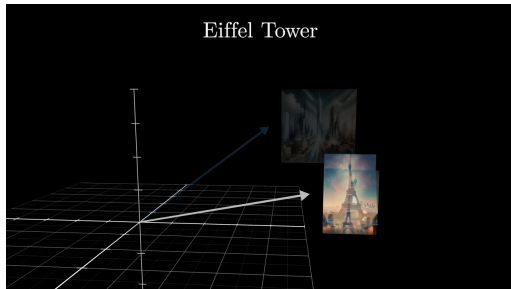
Take a biopsy of the mole

mole means three different things – but its starting embedding is identical in all three.

Self-attention is how each word pulls the context it needs from the *same* sentence. Let's build it – slowly.

Screenshot: 3Blue1Brown, CC BY-NC-SA

Eiffel Tower

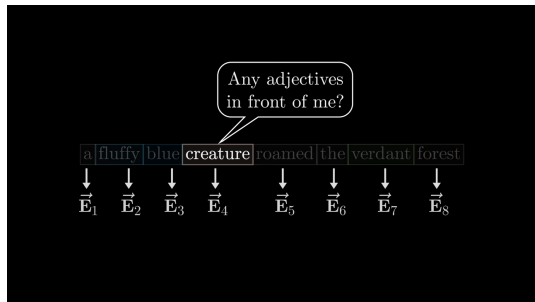


tower, once preceded by Eiffel, should move toward Paris/France/steel.

Self-attention: query, key, value

The chapter's one deep dive. We go slow – what each piece *is*, and *why* the equation looks the way it does.

The goal, made concrete



Screenshot: 3Blue1Brown, CC BY-NC-SA

Sentence: *"a fluffy blue creature roamed the verdant forest."*

We want the noun **creature** to absorb its adjectives **fluffy** and **blue**, so its vector means *"a fluffy, blue creature."*

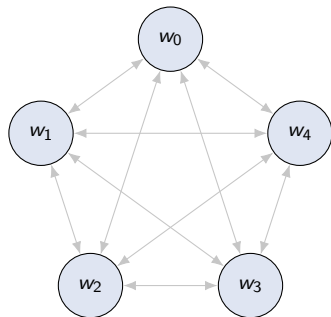
Two questions the mechanism must answer:

- ▶ Which words are **relevant** to creature?
- ▶ **What** should they contribute?

Why “self”-attention? The sentence talks to itself

In L21's translation attention, queries came from the *decoder* and keys/values from the *encoder* – two different sequences.

In **self**-attention, Q , K and V all come from the **same** sentence. Every word at once **asks** (its query) and **answers** (its key and value). One round is **all-to-all messaging**: each word gathers what it needs from every other word.



every word attends to every word

Think of it as **routing**: attention decides who talks to whom, and the values are the messages passed.

The analogy: a soft dictionary lookup

Think of looking something up in a library:

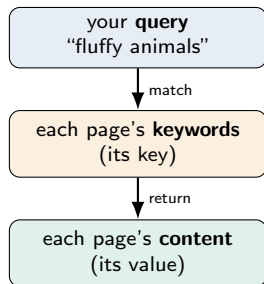
- ▶ You arrive with a **query** – what you want.
- ▶ Each book has a **key** – its label on the spine.
- ▶ Each book has a **value** – its actual contents.

You compare your query to *every* key, then take the **values** of the good matches.

A normal dictionary is a **hard** lookup: one key matches exactly. Attention is a **soft** lookup: *every* key matches a little, weighted by similarity – and you blend all the values.

Every word will produce its own query, key, and value. Next: where these come from.

Another way to picture it: web search

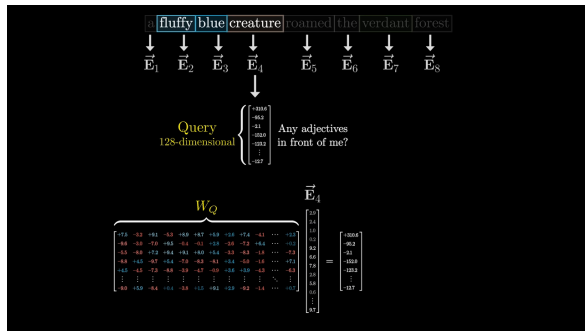


A search engine compares your **query** to every page's **keywords**, ranks the matches, and returns the best pages' **content**.

Attention is the same idea – with one twist: it doesn't return only the top hit. It returns a **blend** of every value, weighted by how well each key matched.

Query = what you search for · **Key** = how a word is indexed · **Value** = what it hands back.

Query — “what am I looking for?”



Multiply a word's embedding by a learned matrix W_Q to get its **query** vector (small – say 128-dim):

$$\mathbf{q}_i = W_Q \mathbf{e}_i.$$

Think of *creature's* query as encoding the question “*are there any adjectives in front of me?*”

W_Q is **learned**. What each word “asks” is discovered from data, not written by us.

Screenshot: 3Blue1Brown, CC BY-NC-SA

Key — “what do I offer?”



A second learned matrix W_K turns each embedding into a **key**:

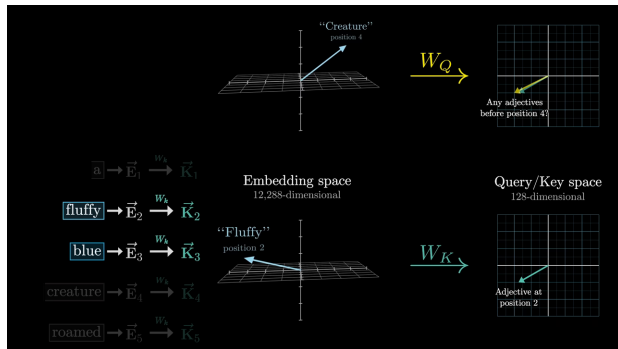
$$\mathbf{k}_j = W_K \mathbf{e}_j.$$

A key *answers* queries. We'd want *fluffy's* key to encode “*I am an adjective, sitting just here.*”

A key is a word *advertising* itself – so that the right queries can find it.

Screenshot: 3Blue1Brown, CC BY-NC-SA

Queries and keys meet in one space



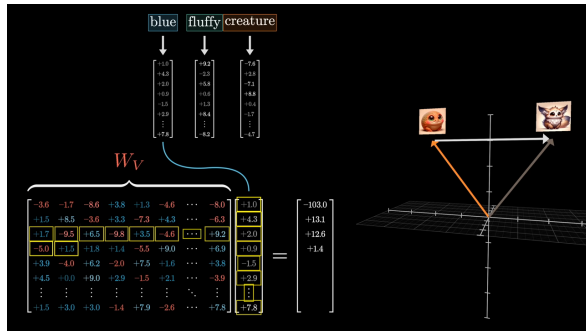
W_Q and W_K **project** the big embedding space down to a shared, smaller **query/key space**.

A good W_Q, W_K arrange it so that *creature*'s query ("adjectives before me?") and *fluffy*'s key ("adjective, here") point the **same way** – so they align.

That is *all* W_Q, W_K do: learn a space where "relevant" means "aligned."

Screenshot: 3Blue1Brown, CC BY-NC-SA

Value — “what do I pass on?”



A third learned matrix W_V makes each word's **value** – the information it will hand over if attended to:

$$\mathbf{v}_j = W_V \mathbf{e}_j.$$

fluffy's value is, roughly, “the fluffiness to add to a noun.”

Key = how you're *found*; value = what you *give*.
They are different jobs, so they get different matrices.

Screenshot: 3Blue1Brown, CC BY-NC-SA

Why three separate matrices?

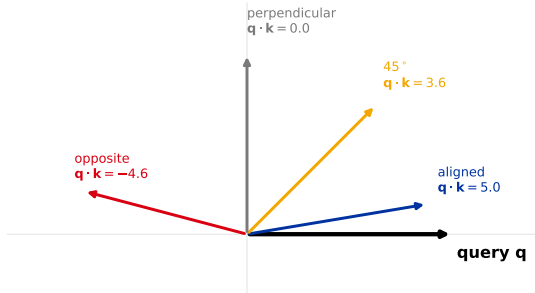
One word plays **three different roles** at once:

Query $\mathbf{q} = W_Q \mathbf{e}$	what I'm <i>looking for</i>	(creature: "adjectives?")
Key $\mathbf{k} = W_K \mathbf{e}$	what I <i>advertise</i>	(fluffy: "I'm an adjective")
Value $\mathbf{v} = W_V \mathbf{e}$	what I <i>contribute</i>	(fluffy: "fluffiness")

Why not just dot the embeddings together directly? Because "*what I'm looking for*" is not the same as "*what I am*" is not the same as "*what I'll give*." Three matrices let the model learn the three roles **independently** – and they are the *only* new weights in attention.

The dot product, up close

Dot product = alignment (big when they point the same way)



The dot product $\mathbf{q} \cdot \mathbf{k} = |\mathbf{q}| |\mathbf{k}| \cos \theta$ is **large** when two vectors point the same way, **zero** when perpendicular, and **negative** when opposite.

So the whole question “how relevant is this key to my query?” collapses to **one number**. Aligned = relevant.

This single operation is the heart of attention. Everything else – softmax, the weighted sum – is bookkeeping around it.

Matching: the dot product scores relevance

	a	fluffy	blue	creature	roamed	the
	$\downarrow \vec{E}_1$	$\downarrow \vec{E}_2$	$\downarrow \vec{E}_3$	$\downarrow \vec{E}_4$	$\downarrow \vec{E}_5$	$\downarrow \vec{E}_6$
	$\downarrow W_Q$	$\downarrow W_Q$	$\downarrow W_Q$	$\downarrow W_Q$	$\downarrow W_Q$	$\downarrow W_Q$
	$\vec{Q}_1$	$\vec{Q}_2$	$\vec{Q}_3$	$\vec{Q}_4$	$\vec{Q}_5$	$\vec{Q}_6$
$\boxed{\text{a}} \rightarrow \vec{E}_1 \xrightarrow{W_k} \vec{K}_1$	$\vec{K}_1 \cdot \vec{Q}_1$	$\vec{K}_1 \cdot \vec{Q}_2$	$\vec{K}_1 \cdot \vec{Q}_3$	$\vec{K}_1 \cdot \vec{Q}_4$	$\vec{K}_1 \cdot \vec{Q}_5$	$\vec{K}_1 \cdot \vec{Q}_6$
$\boxed{\text{fluffy}} \rightarrow \vec{E}_2 \xrightarrow{W_k} \vec{K}_2$	$\vec{K}_2 \cdot \vec{Q}_1$	$\vec{K}_2 \cdot \vec{Q}_2$	$\vec{K}_2 \cdot \vec{Q}_3$	$\vec{K}_2 \cdot \vec{Q}_4$	$\vec{K}_2 \cdot \vec{Q}_5$	$\vec{K}_2 \cdot \vec{Q}_6$
$\boxed{\text{blue}} \rightarrow \vec{E}_3 \xrightarrow{W_k} \vec{K}_3$	$\vec{K}_3 \cdot \vec{Q}_1$	$\vec{K}_3 \cdot \vec{Q}_2$	$\vec{K}_3 \cdot \vec{Q}_3$	$\vec{K}_3 \cdot \vec{Q}_4$	$\vec{K}_3 \cdot \vec{Q}_5$	$\vec{K}_3 \cdot \vec{Q}_6$
$\text{creature} \rightarrow \vec{E}_4 \xrightarrow{W_k} \vec{K}_4$	$\vec{K}_4 \cdot \vec{Q}_1$	$\vec{K}_4 \cdot \vec{Q}_2$	$\vec{K}_4 \cdot \vec{Q}_3$	$\vec{K}_4 \cdot \vec{Q}_4$	$\vec{K}_4 \cdot \vec{Q}_5$	$\vec{K}_4 \cdot \vec{Q}_6$

Score how well key j answers query i with a **dot product** – the same “how-aligned” measure from the embeddings primer:

$$s_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j.$$

Big $\Rightarrow$ “this word is relevant to me.”
Here **fluffy** and **blue** score high against **creature**’s query.

All queries against all keys at once is one matrix product: $S = QK^T$.

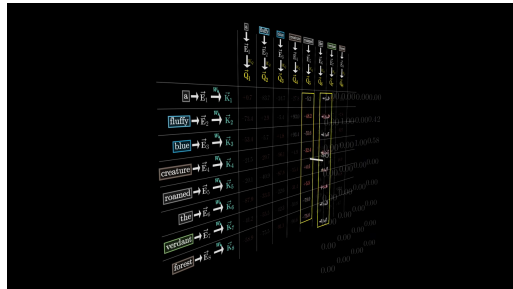
Screenshot: 3Blue1Brown, CC BY-NC-SA

Scores $\rightarrow$ weights: why softmax?

Raw scores can be any real number. But we want a **weighted average** of the values, so the weights must be

- ▶ non-negative, and
- ▶ sum to 1 (a distribution over “where to look”).

Softmax does exactly that – and *softly*: it doesn't just pick the single best key, it spreads weight over all the good ones, and it is smooth (differentiable), so we can train through it.



Screenshot: 3Blue1Brown, CC BY-NC-SA

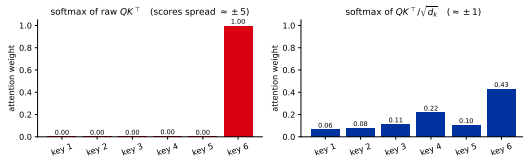
The denominator: why divide by $\sqrt{d_k}$?

A dot product in d_k dimensions is a sum of d_k terms, so its size **grows with** d_k (spread $\sim \sqrt{d_k}$). For $d_k = 64$ the raw scores swing by ± 5 or more.

Feed *big* scores to softmax and it **saturates** – one weight $\rightarrow 1$, the rest $\rightarrow 0$. That is a near-hard argmax: it looks at exactly one word, and its gradient is ≈ 0 , so **training stalls**.

Dividing by $\sqrt{d_k}$ rescales the scores back to $\approx \pm 1$, keeping softmax soft and trainable.

Why $\sqrt{d_k}$: big dimensions make dot products large $\rightarrow$ softmax collapses to a near-hard argmax ($d_k = 64$)



Putting it together: the equation

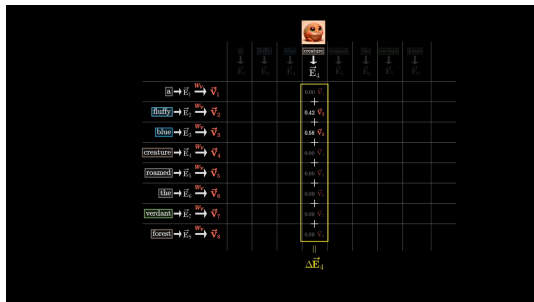
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) V$$

Read it left to right – it is exactly the three moves:

$QK^{\top}$	every query · every key = a grid of relevance scores
$\div \sqrt{d_k}$	keep the scores tame so softmax stays soft
softmax	scores → weights (positive, sum to 1), row by row
$\times V$	weighted sum of the value vectors

“Score, softmax, weighted sum” – the same three moves from the start of the deck, now with learned Q, K, V .

The output: values, weighted and summed



The weights say *how much* to listen; the values say *what* to pass. The output for each word is their weighted sum:

$$\mathbf{o}_i = \sum_j \alpha_{ij} \mathbf{v}_j.$$

creature's output blends big shares of **fluffy's** and **blue's** values – landing on a new point in meaning-space: a *fluffy blue creature*.

Every word is updated this way **at once** – one parallel step. (In the full block this output is **added back** to the word's own vector – a residual connection – so words *refine* rather than overwrite. That's L25.)

Let's run the numbers (1/2)

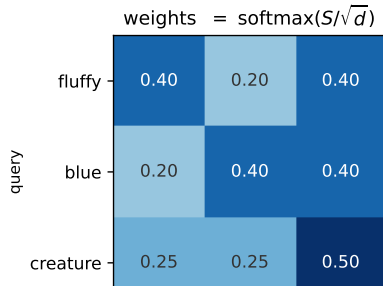
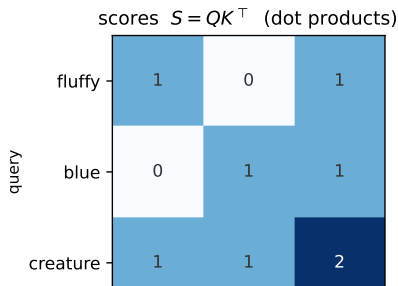
Three words, tiny 2-D embeddings. To keep the arithmetic visible, take $Q = K = V = X$ (skip the learned projections for a moment):

$$\mathbf{e}_{\text{fluffy}} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \mathbf{e}_{\text{blue}} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad \mathbf{e}_{\text{creature}} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Scores $S = QK^\top$ are all the pairwise dot products. *creature's* row:

$$\mathbf{q}_{\text{cre}} \cdot \mathbf{k}_{\text{fluffy}} = 1, \quad \mathbf{q}_{\text{cre}} \cdot \mathbf{k}_{\text{blue}} = 1, \quad \mathbf{q}_{\text{cre}} \cdot \mathbf{k}_{\text{cre}} = 2.$$

A tiny worked example ($Q = K = V = X$, 2-D embeddings)



Let's run the numbers (2/2)

Scale *creature's* scores by $\sqrt{d} = \sqrt{2}$ and softmax:

$$\text{softmax}\left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}, \frac{2}{\sqrt{2}}\right) = (0.25, 0.25, 0.50).$$

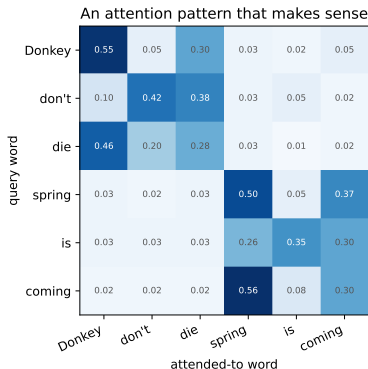
The new *creature* vector is the weighted sum of the values:

$$\mathbf{o}_{\text{cre}} = 0.25\begin{bmatrix} 1 \\ 0 \end{bmatrix} + 0.25\begin{bmatrix} 0 \\ 1 \end{bmatrix} + 0.50\begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.75 \\ 0.75 \end{bmatrix}.$$

creature started at $[1, 1]$ and its attention output is $[0.75, 0.75]$ – pulled a quarter of the way toward **fluffy** and **blue**. **That is attention, by hand.** (Real models insert the learned W_Q, W_K, W_V ; the arithmetic is identical.)

A pattern that makes sense

"Donkey don't die, spring is coming" (illustrative weights)



Our sentence, with the weights laid out (row = query word, column = what it attends to):

- ▶ **coming** looks at **spring** (what is coming?),
- ▶ **die** looks at **Donkey** (who dies?),
- ▶ seasonal words cluster together.

Illustrative weights – hand-set to show the *shape* of a sensible pattern before we look at a real one.

Attention in ~ 15 lines

Every step is a matrix op. In NumPy:

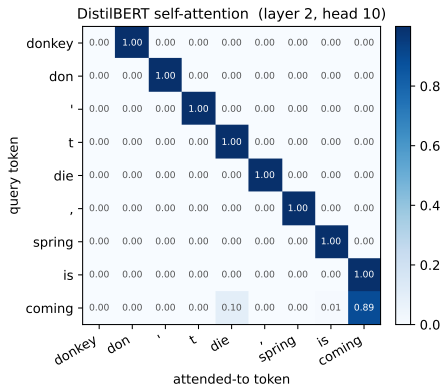
```
import numpy as np

def attention(X, Wq, Wk, Wv):
    Q, K, V = X @ Wq, X @ Wk, X @ Wv          # three learned views
    d_k = Q.shape[1]
    scores = Q @ K.T / np.sqrt(d_k)             # relevance, scaled
    scores -= scores.max(1, keepdims=True)       # numerical stability
    weights = np.exp(scores)
    weights /= weights.sum(1, keepdims=True)     # softmax over each row
    return weights @ V                          # weighted sum of values
```

No loops over positions, no recurrence – three matmuls and a softmax. That simplicity is the whole point.

A real head, our sentence

Real model, our sentence (special tokens dropped)



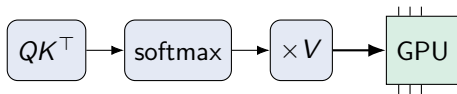
Real attention from a pretrained model (**DistilBERT**) on “*Donkey don't die, spring is coming.*” One of its heads.

This head learned a clean rule: **attend to the next token**. Nobody wrote it – the W_Q , W_K , W_V that produce it were *learned*.

Real heads specialise, and many are less tidy than our illustration. **Different heads do different jobs** – more in L25. (Special tokens dropped for clarity.)

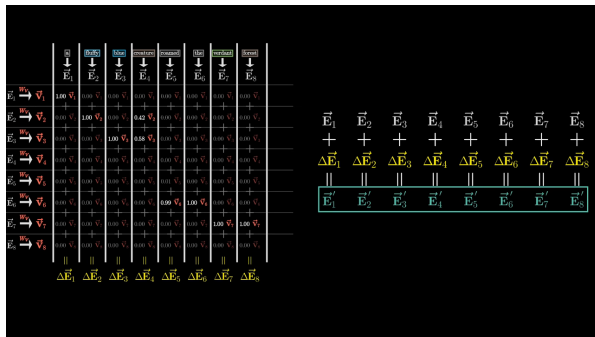
It's just matmul $\rightarrow$ it runs on a GPU

QK^T , softmax, and $\times V$ are all **big matrix multiplies** over the whole sequence at once – no step depends on the previous one.



Exactly the parallel workload a **GPU** eats (L14 “why GPUs,” L16 “a convolution is a matmul”). This is *why* attention scales – and why the RNN, which *couldn't* parallelise over time, lost.

One head of attention, end to end



- Tokens are **vectors**; directions carry meaning.
- Each word emits a **query**, a **key**, a **value** – three learned views.
- **Score** ($QK^\top$), scale ($\sqrt{d_k}$), **softmax**, **weighted sum** of values.
- $e' = e + \Delta e$: the word moves to a context-rich point.
- It's matmul $\rightarrow$ GPU $\rightarrow$ scale.

Next (L25): run many heads at once, put word order back in, and wrap it all in a repeatable block – the Transformer.

Screenshot: 3Blue1Brown, CC BY-NC-SA

Sources & further reading

- ▶ Bahdanau, Cho & Bengio (2015), *Neural Machine Translation by Jointly Learning to Align and Translate* – attention, for translation.
- ▶ Xu et al. (2015), *Show, Attend and Tell* – attention for image captioning.
- ▶ Vaswani et al. (2017), *Attention Is All You Need* – self-attention, the Transformer.
- ▶ Mikolov et al. (2013), word2vec – word embeddings and analogies.
- ▶ Jay Alammar, *The Illustrated Transformer*; Andrej Karpathy, “Let’s build GPT.”
- ▶ **3Blue1Brown**, “Deep Learning” Ch. 5–6 – the visual intuition this deck leans on (screenshots used under CC BY-NC-SA).

For the full, animated story of attention, watch 3Blue1Brown Ch. 6 – it is the clearest explanation out there.